Smart Dine Deployment Guide

This guide covers deploying Smart Dine to production environments.

Prerequisites

- Docker and Docker Compose (recommended)
- Or: Server with Python 3.11+, Node.js 20+, PostgreSQL
- Domain name with SSL certificate
- Reverse proxy (nginx recommended)

Quick Deploy with Docker Compose

1. Environment Setup

Create a `docker-compose.yml`:

```yaml
version: '3.8'

services:
  db:
    image: postgres:15
    environment:
      POSTGRES_DB: smartdine
      POSTGRES_USER: smartdine
      POSTGRES_PASSWORD: your_password
    volumes:
      - postgres_data:/var/lib/postgresql/data
    ports:
      - "5432:5432"

  backend:
    build:
      context: ./backend
      dockerfile: Dockerfile
    environment:
      DATABASE_URL: postgresql://smartdine:your_password@db:5432/smartdine
      SECRET_KEY: your-secret-key-here
      CORS_ORIGINS: https://yourdomain.com
      FRONTEND_PUBLIC_URL: https://yourdomain.com
    ports:
      - "8000:8000"
    depends_on:
      - db
    volumes:
      - ./backend/uploads:/app/uploads

  frontend:
    build:
      context: ./frontend
      dockerfile: Dockerfile
    environment:
      VITE_BACKEND_URL: https://yourdomain.com/api
    ports:
      - "80:80"
```

2. Dockerfiles

backend/Dockerfile:

```dockerfile
FROM python:3.11-slim

WORKDIR /app

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY . .

EXPOSE 8000

CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "8000"]
```

frontend/Dockerfile:

```
FROM node:20-alpine as build

WORKDIR /app

COPY package*.json ./
RUN npm ci

COPY . .
RUN npm run build

FROM nginx:alpine

COPY --from=build /app/dist /usr/share/nginx/html

EXPOSE 80

CMD ["nginx", "-g", "daemon off;"]
```

3. Environment Variables

Create `.env` files for each service with production values:

backend/.env:

```
DATABASE_URL=postgresql://smartdine:prod_password@db:5432/smartdine
SECRET_KEY=your-256-bit-secret-key
CORS_ORIGINS=https://yourdomain.com
FRONTEND_PUBLIC_URL=https://yourdomain.com
ENVIRONMENT=production
```

frontend/.env:

```
VITE_BACKEND_URL=https://yourdomain.com/api
```

4. SSL Configuration

Use certbot or your SSL provider to get certificates, then configure nginx:

```
server {
    listen 443 ssl http2;
    server_name yourdomain.com;

    ssl_certificate /path/to/cert.pem;
    ssl_certificate_key /path/to/key.pem;

    location / {
        proxy_pass http://frontend:80;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
    }

    location /api {
        proxy_pass http://backend:8000;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }

    location /ws {
        proxy_pass http://backend:8000;
        proxy_http_version 1.1;
        proxy_set_header Upgrade $http_upgrade;
        proxy_set_header Connection "upgrade";
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }
}
```

Manual Deployment

Backend Deployment

1. **Server Setup:**

   ```
   # Install Python 3.11
   sudo apt update
   sudo apt install python3.11 python3.11-venv postgresql postgresql-contrib

   # Create database
   sudo -u postgres createdb smartdine
   sudo -u postgres createuser smartdine
   sudo -u postgres psql -c "ALTER USER smartdine PASSWORD 'your_password';"
   ```

2. **Application Setup:**

```
cd /var/www/smartdine/backend
python3.11 -m venv venv
source venv/bin/activate
pip install -r requirements.txt

# Copy and configure .env
cp .env.example .env
nano .env  # Edit with production values

# Initialize database
python init_db.py
python scripts/seed_smart_dine.py
```

3. **Process Manager (systemd):**
   Create `/etc/systemd/system/smartdine-backend.service`:

```
[Unit]
Description=Smart Dine Backend
After=network.target

[Service]
User=www-data
Group=www-data
WorkingDirectory=/var/www/smartdine/backend
Environment=PATH=/var/www/smartdine/backend/venv/bin
ExecStart=/var/www/smartdine/backend/venv/bin/uvicorn app.main:app --host 127.0.0.1 --port 8000
Restart=always

[Install]
WantedBy=multi-user.target
```

```
sudo systemctl enable smartdine-backend
sudo systemctl start smartdine-backend
```

Frontend Deployment

1. **Build:**

```
cd frontend
npm ci
npm run build
```

2. **Serve with nginx:**

```
sudo cp -r dist/* /var/www/html/
sudo chown -R www-data:www-data /var/www/html
```

Database Migration

For production updates:

```
# Backup database first!
pg_dump smartdine > backup.sql

# Run migrations
cd backend
source venv/bin/activate
alembic upgrade head
```

Monitoring & Maintenance

Health Checks

- API health: `GET /api/health`
- Database connectivity check
- WebSocket connection monitoring

Logs

- Backend: Check systemd logs `journalctl -u smartdine-backend`
- Frontend: nginx access/error logs
- Database: PostgreSQL logs

Backups

```
# Database backup
pg_dump smartdine > smartdine_$(date +%Y%m%d_%H%M%S).sql

# File uploads backup
tar -czf uploads_backup.tar.gz backend/uploads/
```

Performance Tuning

PostgreSQL:

```
-- Create indexes for common queries
CREATE INDEX idx_orders_status ON orders(status);
CREATE INDEX idx_orders_created_at ON orders(created_at);
CREATE INDEX idx_order_items_order_id ON order_items(order_id);
```

Backend:

- Use gunicorn with workers: `gunicorn app.main:app -w 4 -k uvicorn.workers.UvicornWorker`
- Enable connection pooling
- Configure proper timeouts

Security Considerations

1. **Environment Variables:**
    - Never commit secrets to version control
    - Use strong, unique SECRET_KEY
    - Rotate keys regularly
2. **Database:**
    - Use strong passwords
    - Restrict database user permissions
    - Enable SSL connections
3. **Network:**
    - Use HTTPS everywhere
    - Configure firewall rules
    - Rate limiting on API endpoints
4. **Updates:**
    - Keep dependencies updated
    - Monitor for security vulnerabilities
    - Regular backup testing

Scaling

Horizontal Scaling

- Multiple backend instances behind load balancer
- Redis for session storage and caching
- Database read replicas

Vertical Scaling

- Increase server resources
- Optimize database queries
- Implement caching layers

Troubleshooting

Common Issues

1. **WebSocket Connections Failing:**
    - Check nginx proxy configuration for WebSocket upgrade headers
    - Verify CORS settings
2. **File Upload Issues:**
    - Check upload directory permissions
    - Verify nginx client_max_body_size
3. **Database Connection Pool Exhausted:**
    - Increase connection pool size
    - Check for connection leaks in code
4. **Slow Performance:**
    - Add database indexes
    - Enable query logging and optimization
    - Implement caching

For additional support, check the application logs and API documentation.